LAB 5 REPORT

Sequences – Lists and Tuples

Student name	Nguyễn Viết Huy
Student ID	ITDSIU24021
Class / Group	IT149IU – Group 01
Course	IT149IU – Fundamentals of Programming (Python)
Submission date	26 October 2026
Included tasks	Tasks 1–9 (required), Tasks 10–11 (optional challenges)

Fundamentals of Programming (Python) · International University – VNU-HCM

AI use note: ChatGPT supported interpretation, code drafting/debugging, and output checking. The solutions should be reviewed and understood before submission.

1. Objectives and theory

This lab practices Python sequences, especially lists and tuples. Core objectives are indexing/slicing, list methods, duplicate elimination, insertion sort, tuples as records, sorting/searching with keys, list queues, list comprehensions and functional-style operations, two-dimensional lists, and using sequences to analyze data.

Important concepts

Lists are mutable sequences; elements may be appended, removed or reassigned. Tuples are immutable sequences suitable for fixed records such as invoices. Slicing uses [start:stop:step], where stop is excluded. A stack is last-in-first-out (LIFO), while a queue is first-in-first-out (FIFO). List comprehensions, lambdas, map/filter/reduce and nested lists provide concise ways to transform and analyze data.

2. Algorithms and solutions

Task 1. List slicing

Algorithm: Build numbers 1 through 20 and use indexes/slices: [2], [:5], [:10], [-5:], [::2], [::-1], and [-3].

Verification / result: The result checks basic indexing, positive and negative indexing, and slice steps.

```
Numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
a) Third number: 3
b) First five: [1, 2, 3, 4, 5]
c) First half: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
d) Last five: [16, 17, 18, 19, 20]
e) Every other number: [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
...
```

Task 2. Duplicate elimination

Algorithm: Visit items from left to right. Append an item only when it is not already in a result list.

Verification / result: Preserves the order of first appearance; tests also cover an empty list and repeated identical values.

```
Original: [23, 34, 67, 87, 22, 43, 34, 87, 23, 33, 22, 34]
Unique: [23, 34, 67, 87, 22, 43, 33]
Empty list test: []
All-same test: [5]
```

Task 3. Insertion sort

Algorithm: For each element from index 1, keep its key, shift larger earlier elements right, and insert the key into the gap.

Verification / result: Tested on the list provided by the handout and on a reproducible random sample. The provided list contains a repeated 23 despite describing its values as unique; the algorithm correctly handles duplicates.

```
Unsorted: [23, 34, 67, 87, 22, 43, 33, 99, 23, 77]
Sorted: [22, 23, 23, 33, 34, 43, 67, 77, 87, 99]
```

Task 4. Guessing game

Algorithm: Store actor/character pairs. Ask for the actor matching a displayed character; continue asking for that character until the answer is correct, then move on.

Verification / result: Case-insensitive matching is supported. The notebook uses a reproducible demonstration to avoid blocking “Run all”; Task4.py provides the interactive game.

```
Who played Ned Stark?
Options: Sean Bean, Mark Addy, Nikolaj Coster-Waldau, Michelle Fairley, Lena Headey
Actor name: Correct!
```

```
Who played Robert Baratheon?
Options: Sean Bean, Mark Addy, Nikolaj Coster-Waldau, Michelle Fairley, Lena Headey
...
```

Task 5. Invoice tuples

Algorithm: Sort invoice tuples by description and unit price with operator.itemgetter; transform to description/quantity and description/invoice-value pairs; filter values from $200 to $500 and sum all invoices.

Verification / result: Prices are floats because the task says to use float for simplicity; values are rounded for display.

```
a) By description:
('83', 'Electric sander', 7, 57.98)
('77', 'Hammer', 76, 11.99)
('39', 'Jig saw', 3, 79.5)
('24', 'Power saw', 18, 99.99)
('7', 'Sledge hammer', 11, 21.5)
...
```

Task 6. User IDs

Algorithm: Generate 8,000 IDs using one letter from a–f and two digits from 1–4, remove duplicates, and sort descending.

Verification / result: There are at most 6 × 4 × 4 = 96 possible IDs, so the deduplicated list cannot exceed 96.

```
Generated IDs: 8000
Unique IDs: 96
Maximum possible: 96
Sorted unique IDs: ['f44', 'f43', 'f42', 'f41', 'f34', 'f33', 'f32', 'f31', 'f24', 'f23', 'f22', 'f21', 'f14', 'f13',
```

Task 7. Queue simulation

Algorithm: Enqueue 3, 2, 1 with append and dequeue with pop(0). Repeat with collections.deque and popleft.

Verification / result: The observed order is 3, 2, 1. list.pop(0) moves all later elements (O(n)); deque.popleft is approximately O(1).

```
List queue: [3, 2, 1]
dequeue: 3
dequeue: 2
dequeue: 1
deque popleft: 3
deque popleft: 2
...
```

Task 8. Daily infection statistics

Algorithm: Use statistics to report min, max, range, mean, median, sample variance and sample standard deviation, with population measures for comparison.

Verification / result: The bar chart has day on the x-axis and new daily cases on the y-axis. Range is max minus min.

```
Minimum: 174
Maximum: 1704
Range: 1530
Mean: 845.70
Median: 704.50
Sample variance: 253620.75
...
```

Task 9. Two-dice simulation

Algorithm: For 360, 36,000 and 36,000,000 trials, roll two random integers from 1 to 6, count sums 2–12 and calculate percentages.

Verification / result: NumPy vectorized chunking keeps memory usage manageable; count totals are checked against N. Sum 7 is expected to be most common, while 2 and 12 are least common.

```
Rolls: 360
Sum | Count | Percent
  2 |         7 |    1.944%
  3 |        19 |    5.278%
  4 |        36 |   10.000%
  5 |        42 |   11.667%
...
```

Task 10. Matrix operations (optional)

Algorithm: Implement transpose with a list comprehension and zip(*m); add matrices with nested loops; multiply using three nested loops.

Verification / result: Self-check A×B equals [[58, 64], [139, 154]]. NumPy is not used for matrix operations.

```
transpose(A):
     1      4
     2      5
     3      6
A + D:
    11     22     33
...
```

Task 11. Balanced brackets (optional)

Algorithm: Push opening brackets onto a list stack. For each closing bracket, fail if the stack is empty or its top does not match; at the end the stack must be empty.

Verification / result: Checks the task examples plus empty input, no-bracket input, nested valid brackets and mismatched brackets.

```
'(a[b]{c})' -> True
'([)]' -> False
'((x)' -> False
'' -> True
'no brackets' -> True
'{[()()]}' -> True
...
```

3. Summary: what I learned

This lab reinforced how to choose and manipulate Python sequence types. I practiced expressing ranges with slicing, preserving order while removing duplicates, implementing insertion sort manually, storing structured records in tuples, sorting records using keys, and modeling FIFO queues. The data-science exercises connected lists with descriptive statistics, charts and simulation. The challenge tasks extended these ideas to matrix algorithms and stack-based bracket validation.

4. Submission checklist

☐ Completed notebook with code cells executed top-to-bottom.

☐ One standalone .py file per task, with student, ID, course, lab, date and description comments.

☐ Output evidence files in outputs/ (console transcripts in PNG/TXT, plus charts).

☐ This PDF report and a README with run instructions and notes.

☐ AI assistance disclosed in the README and report.